

1.2. Dadas las siguientes historias (schedules):

$H_1 = r_1(X); r_4(Y); w_1(X); w_4(X); r_3(X); c_1; w_4(Y); w_3(Y); w_4(Z); c_4; w_3(X); c_3.$

$H_2 = r_1(X); w_2(X); r_1(Y); w_1(X); w_1(Y); c_1; r_2(Z); w_2(Y); c_2.$

$H_3 = w_1(X); w_2(X); w_2(Y); c_2; w_1(Y); c_1; w_3(X); w_3(Y); c_3.$

$H_4 = w_2(X); r_1(X); r_2(Z); r_1(Y); w_2(Y); w_1(Y); c_2; w_1(X); c_1.$

$H_5 = r_1(X); r_4(Y); r_3(X); w_1(X); w_4(Y); w_3(X); c_1; w_3(Y); w_4(Z); c_3; W_4(X); c_4.$

(a) Indicar para cada una si es NoRC, RC, ACA o ST.

(b) Indicar cuáles podrían producir un *dirty read* o un *lost update*.

Historia recuperable : RC

H es RC si siempre fue una tx
 T_i lee de T_j con $i \neq j$ en H y
 $C_i \in H \Rightarrow C_j < C_i$

H es recuperable si una tx realiza Commit
sólo después de que iniciaron Commit
Todas las tx de las cuales lee.

Avoid Cascading Aborts : ACA

H es ACA si siempre fue una tx
 T_i lee X de T_j con $i \neq j$ en H, $\Rightarrow$
 $C_j < r_i(x)$

Lee sólo valores de tx que ya
iniciaron Commit.

Stricto : ST

H es ST si siempre fue $w_j(x) < o_i(x)$
 $i \neq j$
entonces $a_j < o_i(x)$ o $C_j < o_i(x)$

$$\text{viendo } O_i(x) = \tau_i(x) \\ = w_i(x)$$

C_j : Commit
 a_j : Abort

Es decir no se puede leer ni escribir un item hasta que la tx fue lo escribió previamente haya hecho Commit / Abort.

$$ST \subset ACA \subset RC$$

SR interseca a todos, es ortogonal

Volvamos al 1.2:

H₁:

T ₁	T ₂	T ₃	T ₄
r ₁ (x)			
			r ₄ (y)
w ₁ (x)			
			w ₄ (x)
		r ₃ (x)	
c ₁			
			w ₄ (y)
		w ₃ (y)	
			w ₄ (z)
			c ₄
		w ₃ (x)	
		c ₃	

→ ST no es, pues w₄(x), pero w₁(x) es anterior y no hizo commit.

→ ACA no es, pues r₃(x) lee x, pero T₄ ni T₁ hicieron commit

→ RC: es RC, pues T₃ lee de T₄ e hizo commit antes.

Se puede producir dirty read, pues T_3 lee x de T_4 y esto aún no hizo commit

Se puede producir lost Update, pues T_4 escribe x sin leer el valor anterior.

H₂:

T_1	T_2
$r_1(x)$	
	$w_2(x)$
$r_1(y)$ $w_1(x)$ $w_1(y)$ c_1	
	$r_2(z)$ $w_2(y)$ c_2

⇒ Es RC ✓

Hace commit luego de que hicieron commit todos los tx de los que lee.
(no lee de nadie)

⇒ Es ACA? ✓

Lee valores de tx que ya hicieron commit

⇒ Es ST? X

Porque T_2 hace $w_2(x)$ antes fue T_1 , pero T_1 hace $w_1(x)$... commit
antes fue T_2

⇒ No hay Posibles Dirty-read

⇒ Hay posibles lost-update?

$w_1(x)$ es posible lost update?
hubo en $w_2(x)$, $w_1(x)$

$w_2(x)$ no es lost update? Noche
después finalmente

<u>H3</u>		
T_1	T_2	T_3
$w_1(x)$		
	$w_2(x)$ $w_2(y)$ c_2	
$w_1(y)$ c_1		$w_3(x)$ $w_3(y)$ c_3

RC? ✓
No hay lecturas

ACA? ✓
No hay lect.

ST X
 T_2 hace c_2
antes que T_1
y T_1 hizo w_1
antes.

Dirty-read? No,
no hay lecturas

$w_1(y)$?
Podría serlo?
 T_2 ya hizo commit

lost-update? Si, $w_2(x)$
y justo antes hubo
 $w_1(x)$ sin commit

H₂

T ₁	T ₂
	w ₂ (x)
r ₁ (x)	
	r ₂ (z)
r ₁ (y)	
	w ₂ (y)
w ₁ (y)	
	c ₂
w ₁ (x) c ₁	

RC: ✓

Before T₁ lee de
T₂ y T₂ lee
Commit antes

ACA: ✗

Pues T₁ lee
r₁(x), T₂ lee
w₂(x) y
aún no lee
Commit.

Dirty-read:

Hay posibilidad
pues r₁(x) lee
de T₂ sin
pueda hacer
Commit

lost-update: sí,

pues w₁(y) y
antes hubo w₂(y)
write sin
read justo
antes.

H₃:

T_1	T_2	T_3	T_4
$r_1(x)$			
			$c_4(y)$
		$r_3(x)$	
$w_1(x)$			
			$w_4(y)$
		$w_3(x)$	
c_1		$w_3(y)$	
			$w_4(z)$
		c_3	
			$w_4(x)$ c_4

RC: ✓ Todas las lecturas son de T1x
y e commit.

ACA: ✓ " "

ST: x $w_3(x)$ antes de T_1 hace commit
Dirty-read: No, nadie lee

lost-update: α , pues $w_3(x)$ y justo
antes $w_1(x)$ y T_3 no hizo $R_3(x)$.